

Advanced Data Management

Chapter 1 — Class Examples (live demos)

Cycle Ingénieur · S3

Fall 2026

These are examples to run *live* in class, on PostgreSQL. Each one has the commands, a sample of the output, and a note on what to highlight. They use the `adm_lab` database built in Lab 1; a compact fallback schema is given below so the examples also run on their own.

1. Setup (fallback schema)

If the Lab 1 database is not loaded, create a tiny version:

```
CREATE TABLE customers (id serial PRIMARY KEY, name text, country char
(2));
CREATE TABLE orders (id serial PRIMARY KEY, cust_id int REFERENCES
customers(id),
amount numeric(8,2), order_date date);

INSERT INTO customers (name, country)
SELECT 'Customer ' || g, (ARRAY['MA','FR','ES','US','DE'])[1+floor(
random()*5)::int]
FROM generate_series(1,10000) g;

INSERT INTO orders (cust_id, amount, order_date)
SELECT 1+floor(random()*10000)::int, round((random()*100+5)::numeric,2),
date '2025-01-01' + (floor(random()*365)::int)
FROM generate_series(1,50000) g;

ANALYZE; -- refresh catalog statistics (important for the optimizer)
```

2. Example 1 — The life of a query, on a real engine

Run our lecture query and ask the engine to *show its plan* instead of results:

```
EXPLAIN
SELECT c.name, SUM(o.amount) AS total
FROM customers c
JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA'
GROUP BY c.name;
```

Sample output (yours will differ slightly):

```
HashAggregate (cost=... rows=...)
Group Key: c.name
-> Hash Join (cost=...)
```

```

Hash Cond: (o.cust_id = c.id)
-> Seq Scan on orders o  (cost=...)
-> Hash  (cost=...)
    -> Seq Scan on customers c  (cost=...)
        Filter: (country = 'MA'::bpchar)

```

What to point out. This tree *is* the physical plan from the lecture. Read it bottom-up: scan customers (with the `country='MA'` filter), scan orders, hash-join them, then aggregate. Map each node back to the five stages — the optimizer produced this; the executor runs it.

Now add `ANALYZE` to actually run it and see real timings and row counts:

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA' GROUP BY c.name;

```

What to point out. Compare `rows` (estimated) with `actual rows`. When they are close, the optimizer’s statistics are good — a theme we return to in Chapter 5. `BUFFERS` previews the buffer pool (Chapter 6).

3. Example 2 — Declarative vs. procedural: one query, two plans

The same SQL can run different ways. Force the engine’s hand to make the point:

```

EXPLAIN SELECT * FROM orders WHERE amount > 100;      -- plan A

SET enable_seqscan = off;                               -- discourage full
scans
CREATE INDEX idx_amount ON orders(amount);
EXPLAIN SELECT * FROM orders WHERE amount > 100;      -- plan B
RESET enable_seqscan;
DROP INDEX idx_amount;

```

What to point out. The SQL text never changed — only the *how*. You (declaratively) asked for rows with `amount > 100`; the system chose a strategy. This is exactly the “you say what, the DBMS decides how” idea, made visible. (Indexes themselves are Chapter 3.)

4. Example 3 — OLTP vs. OLAP query shapes

```

-- OLTP shape: one tiny record, by key
SELECT * FROM customers WHERE id = 4275;

-- OLAP shape: scan many rows, aggregate a column
SELECT c.country, COUNT(*) AS orders, ROUND(AVG(o.amount),2) AS avg_amt
FROM customers c JOIN orders o ON o.cust_id = c.id
GROUP BY c.country
ORDER BY orders DESC;

```

What to point out. The first touches one row and returns instantly; the second sweeps tens of thousands and aggregates. Same database, two *workloads* — and later, two storage choices (row vs. column).

5. Example 4 — Row vs. column, felt directly (DuckDB)

Export the orders and run the same aggregation in a columnar engine:

```
# in a terminal, using the DuckDB CLI or Python
COPY (SELECT * FROM orders) TO '/tmp/orders.parquet' (FORMAT parquet);
-- from psql

-- in DuckDB
SELECT AVG(amount) FROM '/tmp/orders.parquet';
```

What to point out. DuckDB reads only the `amount` column from the Parquet file, not whole rows. On a wide table the difference is dramatic — the concrete reason column stores win for analytics.

6. Example 5 — SQL refresher, live

```
-- join + top-N
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.cust_id = c.id
GROUP BY c.name ORDER BY total DESC LIMIT 5;

-- WHERE filters rows, HAVING filters groups
SELECT country, COUNT(*) FROM customers
GROUP BY country HAVING COUNT(*) > 1800;

-- subquery
SELECT name FROM customers
WHERE id IN (SELECT cust_id FROM orders WHERE amount > 100);
```

What to point out. Run each, then ask the class to predict the plan before you `EXPLAIN` it. Reinforces the map between SQL clauses and plan operators.

7. Example 6 — The catalog is just data

```
-- tables and their sizes
SELECT relname, reltuples::bigint AS est_rows
FROM pg_class WHERE relkind = 'r' AND relname IN ('customers', 'orders');

-- statistics the optimizer uses
SELECT attname, n_distinct, null_frac
FROM pg_stats WHERE tablename = 'orders';
```

What to point out. The database describes itself in ordinary tables you can query. These very numbers (`reltuples`, `n_distinct`) are what the optimizer reads to estimate cost — closing the loop with Example 1.

These examples pair with the Chapter 1 slides and Lab 1. Costs and row counts vary by machine and by the random data generated; the *shapes* of the plans are the point.